

LAPORAN PRAKTIKUM
STRUKTUR DATA
“IMPLEMENTASI SINGLE LINKED LIST DALAM BAHASA
PEMROGRAMAN JAVA”

disusun oleh

Hafizh Habibullah

2511531002

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

MUHAMMAD GALID AVERO



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Allah SWT. karena atas rahmat dan karunia-Nya laporan Praktikum Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum pada mata kuliah Struktur Data dengan topik Implementasi Single Linked List dalam Bahasa Pemrograman Java. Melalui laporan ini, penulis berusaha menjelaskan hasil praktikum yang meliputi penjelasan mengenai kode program, tujuan, serta cara kerja program.

Penulis sangat mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di kemudian hari. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 6 Mei 2026

Hafizh Habibullah

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	3
1.3 Manfaat.....	3
BAB 2 PEMBAHASAN.....	5
2.1 Implementasi Node pada Single Linked List.....	5
2.2 Implementasi Penambahan Data pada Single Linked List	5
2.2.1 Menambahkan Node di Depan	5
2.2.2 Menambahkan Node di Belakang	6
2.2.3 Membuat Node Baru	6
2.2.4 Menambahkan Node di Posisi Tertentu.....	7
2.2.5 Menampilkan Isi Linked List	7
2.2.6 Method Main.....	8
2.3 Implementasi Pencarian Data pada Single Linked List	9
2.3.1 Mencari Data Tertentu.....	9
2.3.2 Traversal.....	9
2.3.3 Method Main.....	10
2.4 Implementasi Penghapusan Data pada Single Linked List.....	10
2.4.1 Menghapus Node Terdepan.....	10
2.4.2 Menghapus Node Terakhir	11
2.4.3 Menghapus Node di Posisi Tertentu.....	12
2.4.4 Menampilkan Linked List	12
2.4.5 Method Main.....	13
2.5 Latihan Pasien dan Rumah Sakit	14
BAB III KESIMPULAN.....	21
3.1 Kesimpulan.....	21
DAFTAR PUSTAKA	22

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pengembangan program karena berpengaruh terhadap efisiensi penyimpanan dan pengolahan data. Salah satu struktur data linear yang sering digunakan adalah *Linked List*, yaitu struktur data yang terdiri dari kumpulan *node* yang saling terhubung melalui *pointer* atau referensi.

Salah satu jenis *Linked List* yang paling dasar adalah *Single Linked List* (SLL). Pada struktur ini, setiap *node* hanya memiliki satu *pointer* yang mengarah ke *node* berikutnya. Dengan konsep tersebut, proses penambahan dan penghapusan data dapat dilakukan secara dinamis tanpa harus menentukan ukuran data di awal, seperti pada Array.

Implementasi *Single Linked List* memiliki beberapa operasi dasar seperti menambah data, menampilkan data, mencari data, dan menghapus data tertentu. Struktur ini banyak digunakan karena lebih fleksibel dalam pengelolaan data yang ukurannya dapat berubah sewaktu-waktu. Oleh karena itu, diperlukan pemahaman mengenai konsep dan implementasi *Single Linked List* agar dapat digunakan secara efektif dalam pembuatan program.

1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar struktur data *Single Linked List*.
2. Memahami cara kerja *node* dan *pointer* pada *Single Linked List*.
3. Mengimplementasikan operasi dasar *Single Linked List* menggunakan bahasa pemrograman Java.
4. Melatih kemampuan dalam mengelola data secara dinamis menggunakan struktur data *linked list*.

1.3 Manfaat

Manfaat dari Praktikum ini adalah:

1. Menambah pemahaman mengenai struktur data dinamis, khususnya Single Linked List.
2. Melatih kemampuan logika pemrograman dalam pengolahan data menggunakan Linked List.
3. Memahami proses penambahan, pencarian, penampilan, dan penghapusan data pada Single Linked List.
4. Meningkatkan kemampuan implementasi struktur data dalam pemrograman Java.

BAB 2

PEMBAHASAN

2.1 Implementasi Node pada Single Linked List

```
1 package pekan5_2511531002;
2
3 public class NodeSLL_2511531002 {
4     int data_1002;
5     NodeSLL_2511531002 next_1002;
6
7     public NodeSLL_2511531002(int data_1002) {
8         this.data_1002 = data_1002;
9         this.next_1002 = null;
10    }
11 }
```

Kode Program 2. 1 – Program Node pada Single Linked List

Class **NodeSLL_2511531002** digunakan untuk merepresentasikan *node* pada struktur data Single Linked List. Setiap node berfungsi untuk menyimpan sebuah data serta referensi yang mengarah ke node berikutnya.

Pada class ini terdapat atribut **data_1002** yang bertipe integer dan digunakan untuk menyimpan nilai data. Selain itu, terdapat atribut **next_1002** yang digunakan sebagai pointer atau penghubung ke node selanjutnya dalam linked list.

Class ini juga memiliki konstruktor **NodeSLL_2511531002(int data_1002)** yang berfungsi untuk menginisialisasi data pada node saat objek dibuat. Nilai **next_1002** diatur **null** sebagai penanda bahwa node belum terhubung ke node lain.

2.2 Implementasi Penambahan Data pada Single Linked List

2.2.1 Menambahkan Node di Depan

```
1 package pekan5_2511531002;
2
3 public class TambahSLL_2511531002 {
4     public static NodeSLL_2511531002 insertAtFront_2511531002(NodeSLL_2511531002 head_1002, int value_1002) {
5         NodeSLL_2511531002 new_node_1002 = new NodeSLL_2511531002(value_1002);
6         new_node_1002.next_1002 = head_1002;
7         return new_node_1002;
8     }
9 }
```

Kode Program 2.2 – Blok Program Menambahkan Node di Depan

Method ini digunakan untuk menambahkan node baru di bagian depan linked list. Pertama, method membuat objek node baru menggunakan nilai yang dimasukkan. Setelah itu, pointer **next_1002** dari node baru diarahkan ke head lama sehingga node lama tetap terhubung. Kemudian method mengembalikan node baru sebagai head yang baru.

2.2.2 Menambahkan Node di Belakang

```
10 public static NodeSLL_2511531002 insertAtEnd_2511531002(NodeSLL_2511531002 head_1002, int value_1002) {
11     NodeSLL_2511531002 newNode_1002 = new NodeSLL_2511531002(value_1002);
12     if (head_1002 == null) {
13         return newNode_1002;
14     }
15     NodeSLL_2511531002 last_1002 = head_1002;
16     while (last_1002.next_1002 != null) {
17         last_1002 = last_1002.next_1002;
18     }
19     last_1002.next_1002 = newNode_1002;
20     return head_1002;
21 }
22
```

Kode Program 2.3 – Blok Program Menambahkan Node di Belakang

Method ini digunakan untuk menambahkan node baru di bagian akhir linked list. Method awalnya membuat node baru. Jika linked list masih kosong (**head_1002 == null**), maka node baru langsung dijadikan head. Namun jika list sudah memiliki isi, program akan menelusuri node satu per satu menggunakan perulangan hingga mencapai node terakhir yang memiliki nilai **next_1002 = null**. Setelah itu, node terakhir dihubungkan dengan node baru melalui pointer **next_1002**.

2.2.3 Membuat Node Baru

```
23 static NodeSLL_2511531002 GetNode_2511531002(int data_1002) {
24     return new NodeSLL_2511531002(data_1002);
25 }
26
```

Kode Program 2.4 – Blok Program Membuat Node Baru

Method ini digunakan untuk membuat dan mengembalikan node baru berdasarkan data yang diberikan. Method ini mempermudah proses pembuatan node agar dapat digunakan kembali pada method lain.

2.2.4 Menambahkan Node di Posisi Tertentu

```
27 static NodeSLL_2511531002 insertPos_2511531002(NodeSLL_2511531002 headNode_1002, int position_1002, int value_1002) {  
28     NodeSLL_2511531002 head_1002 = headNode_1002;  
29     if (position_1002 < 1)  
30         System.out.print("Invalid position");  
31     if (position_1002 == 1) {  
32         NodeSLL_2511531002 new_node_1002 = new NodeSLL_2511531002(value_1002);  
33         new_node_1002.next_1002 = head_1002;  
34         return new_node_1002;  
35     } else {  
36         while (position_1002-- > 0) {  
37             if (position_1002 == 1) {  
38                 NodeSLL_2511531002 newNode_1002 = GetNode_2511531002(value_1002);  
39                 newNode_1002.next_1002 = headNode_1002.next_1002;  
40                 headNode_1002.next_1002 = newNode_1002;  
41                 break;  
42             }  
43             headNode_1002 = headNode_1002.next_1002;  
44         }  
45         if (position_1002 > 1)  
46             System.out.print("Posisi di luar jangkauan");  
47         return head_1002;  
48     }  
49 }
```

Kode Program 2.5 – Blok Program Menambahkan Node di Posisi Tertentu

Method ini digunakan untuk menambahkan node pada posisi tertentu di linked list. Program terlebih dahulu memeriksa apakah posisi yang dimasukkan valid. Jika posisi bernilai 1, maka node baru akan ditambahkan di depan linked list.

Jika posisi lebih dari 1, program akan menelusuri linked list menggunakan perulangan hingga mencapai posisi sebelum lokasi penyisipan. Setelah posisi ditemukan, node baru dibuat lalu pointer **next_1002** node baru diarahkan ke node berikutnya. Selanjutnya pointer **next_1002** node sebelumnya dihubungkan ke node baru sehingga node berhasil disisipkan pada posisi yang diinginkan.

2.2.5 Menampilkan Isi Linked List

```
51 public static void printList_2511531002(NodeSLL_2511531002 head_1002) {  
52     NodeSLL_2511531002 curr_1002 = head_1002;  
53     while (curr_1002.next_1002 != null) {  
54         System.out.print(curr_1002.data_1002+"→");  
55         curr_1002 = curr_1002.next_1002;  
56     }  
57     if (curr_1002.next_1002 == null) {  
58         System.out.print(curr_1002.data_1002);  
59         System.out.println();  
60     }  
61 }  
62 }
```

Kode Program 2.6 – Blok Program Menampilkan Isi Linked List

Method ini digunakan untuk menampilkan seluruh isi linked list. Program menggunakan variabel bantuan **curr_1002** untuk menelusuri node mulai dari head hingga node terakhir. Setiap data node akan ditampilkan

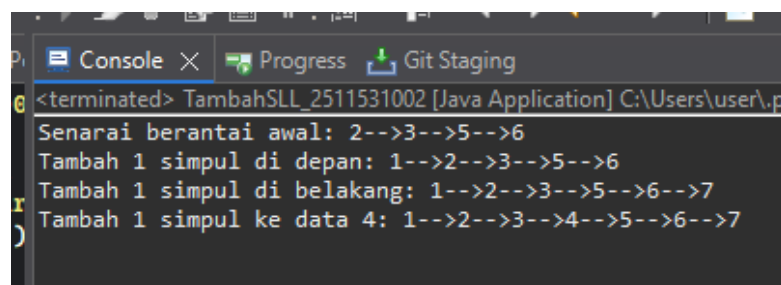
secara berurutan menggunakan format panah (-->) sebagai penghubung antar node.

2.2.6 Method Main

```
63 public static void main(String[] args) {  
64     NodeSLL_2511531002 head_1002 = new NodeSLL_2511531002(2);  
65     head_1002.next_1002 = new NodeSLL_2511531002(3);  
66     head_1002.next_1002.next_1002 = new NodeSLL_2511531002(5);  
67     head_1002.next_1002.next_1002.next_1002 = new NodeSLL_2511531002(6);  
68     System.out.print("Senarai berantai awal: ");  
69     printList_2511531002(head_1002);  
70  
71     System.out.print("Tambah 1 simpul di depan: ");  
72     int data_1002 = 1;  
73     head_1002 = insertAtFront_2511531002(head_1002, data_1002);  
74     printList_2511531002(head_1002);  
75  
76     System.out.print("Tambah 1 simpul di belakang: ");  
77     int data2_1002 = 7;  
78     head_1002 = insertAtEnd_2511531002(head_1002, data2_1002);  
79     printList_2511531002(head_1002);  
80  
81     System.out.print("Tambah 1 simpul ke data 4: ");  
82     int data3_1002 = 4;  
83     int pos_1002 = 4;  
84     head_1002 = insertPos_2511531002(head_1002, pos_1002, data3_1002);  
85     printList_2511531002(head_1002);  
86 }  
87 }
```

Kode Program 2.6 – Blok Program Main

Method ini merupakan method utama yang digunakan untuk menjalankan program. Pada method ini dibuat linked list awal dengan beberapa node. Selanjutnya program melakukan pengujian operasi penambahan node di bagian depan, belakang, dan posisi tertentu menggunakan method yang telah dibuat sebelumnya. Hasil setiap operasi kemudian ditampilkan menggunakan method **printList_2511531002()**, yang dimana Hasil Outputnya adalah sebagai berikut:



```
<terminated> TambahSLL_2511531002 [Java Application] C:\Users\user\p  
Senarai berantai awal: 2-->3-->5-->6  
Tambah 1 simpul di depan: 1-->2-->3-->5-->6  
Tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7  
Tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

Gambar 2. 1 – Hasil Program Penambahan Node pada SLL

2.3 Implementasi Pencarian Data pada Single Linked List

2.3.1 Mencari Data Tertentu

```
1 package pekan5_2511531002;
2
3 public class PencarianSLL_2511531002 {
4     static boolean searchKey_2511531002(NodeSLL_2511531002 head_1002, int key_1002) {
5         NodeSLL_2511531002 curr_1002 = head_1002;
6         while (curr_1002 != null) {
7             if (curr_1002.data_1002 == key_1002)
8                 return true;
9             curr_1002 = curr_1002.next_1002;
10        }
11        return false;
12    }
```

Kode Program 2.7 – Blok Program Mencari Data Tertentu

Method ini digunakan untuk mencari data tertentu pada linked list. Method bekerja dengan menelusuri node mulai dari head menggunakan variabel bantu **curr_1002**. Selama node yang ditelusuri tidak bernilai **null**, program akan membandingkan data pada node dengan nilai **key_1002** yang dicari. Namun, jika penelusuran telah mencapai akhir linked list dan data tidak ditemukan, method akan mengembalikan nilai **false**.

2.3.2 Traversal

```
13
14     public static void traversal_2511531002(NodeSLL_2511531002 head_1002) {
15         NodeSLL_2511531002 curr_1002 = head_1002;
16         while (curr_1002 != null) {
17             System.out.print(" " + curr_1002.data_1002);
18             curr_1002 = curr_1002.next_1002;
19         }
20         System.out.println();
21     }
```

Kode Program 2.8 – Blok Program Traversal

Method ini digunakan untuk menampilkan seluruh data pada linked list secara berurutan. Program melakukan penelusuran mulai dari head hingga node terakhir menggunakan variabel **curr_1002**. Setiap data pada node akan ditampilkan selama node masih terhubung.

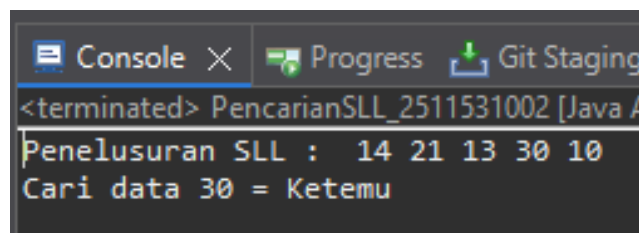
2.3.3 Method Main

```
23 public static void main(String[] args) {
24     NodeSLL_2511531002 head_1002 = new NodeSLL_2511531002(14);
25     head_1002.next_1002 = new NodeSLL_2511531002(21);
26     head_1002.next_1002.next_1002 = new NodeSLL_2511531002(13);
27     head_1002.next_1002.next_1002.next_1002 = new NodeSLL_2511531002(30);
28     head_1002.next_1002.next_1002.next_1002.next_1002 = new NodeSLL_2511531002(10);
29     System.out.print("Penelusuran SLL : ");
30     traversal_2511531002(head_1002);
31
32     int key_1002 = 30;
33     System.out.print("Cari data " + key_1002 + " = ");
34     if (searchKey_2511531002(head_1002, key_1002))
35         System.out.print("Ketemu");
36     else
37         System.out.print("Tidak ada");
38 }
39 }
```

Kode Program 2.9 – Blok Program Main

Method ini merupakan method utama yang digunakan untuk menjalankan program. Pada method ini dibuat beberapa node yang saling terhubung untuk membentuk Single Linked List.

Selanjutnya program menampilkan isi linked list menggunakan method `traversal_2511531002()`. Setelah itu, dilakukan proses pencarian data menggunakan method `searchKey_2511531002()` dengan nilai yang dicari berupa 30. Jika data ditemukan maka program akan menampilkan pesan “Ketemu”, sedangkan jika data tidak ditemukan akan menampilkan pesan “Tidak Ada”. Hasil output programnya adalah sebagai berikut:



Gambar 2. 2 – Hasil Program Pencarian Data pada SLL

2.4 Implementasi Penghapusan Data pada Single Linked List

2.4.1 Menghapus Node Terdepan

```
1 package pekan5_2511531002;
2
3 public class HapusSLL_2511531002 {
4     public static NodeSLL_2511531002 deleteHead_2511531002(NodeSLL_2511531002 head_1002) {
5         if (head_1002 == null) return null;
6         head_1002 = head_1002.next_1002;
7         return head_1002;
8     }
9 }
```

Kode Program 2.10 – Blok Program Menghapus Node Terdepan

Method ini digunakan untuk menghapus node pertama atau head pada linked list. Program terlebih dahulu memeriksa apakah linked list kosong. Jika kosong, method akan mengembalikan nilai **null**.

Jika linked list memiliki isi, maka head akan dipindahkan ke node berikutnya menggunakan **head_1002.next_1002**. Dengan begitu, node pertama sebelumnya tidak lagi terhubung ke linked list dan dianggap terhapus.

2.4.2 Menghapus Node Terakhir

```
9
10 public static NodeSLL_2511531002 removeLastNode_2511531002(NodeSLL_2511531002 head_1002) {
11     if (head_1002 == null) return null;
12     if (head_1002.next_1002 == null) return null;
13     NodeSLL_2511531002 secondLast_1002 = head_1002;
14     while (secondLast_1002.next_1002.next_1002 != null) {
15         secondLast_1002 = secondLast_1002.next_1002;
16     }
17     secondLast_1002.next_1002 = null;
18     return head_1002;
19 }
```

Kode Program 2.11 – Blok Program Menghapus Node Terakhir

Method ini digunakan untuk menghapus node terakhir pada linked list. Program terlebih dahulu memeriksa apakah linked list kosong atau hanya memiliki satu node.

Jika linked list memiliki lebih dari satu node, program akan menelusuri node hingga mencapai node sebelum node terakhir menggunakan variabel **secondLast_1002**. Setelah ditemukan, pointer **next_1002** dari node tersebut diubah menjadi **null** sehingga hubungan ke node terakhir terputus dan node terakhir terhapus.

2.4.3 Menghapus Node di Posisi Tertentu

```
21 public static NodeSLL_2511531002 deleteNode_2511531002(NodeSLL_2511531002 head_1002, int position_1002) {  
22     NodeSLL_2511531002 temp_1002 = head_1002;  
23     NodeSLL_2511531002 prev_1002 = null;  
24     if (temp_1002 == null) return head_1002;  
25     if (position_1002 == 1) {  
26         head_1002 = temp_1002.next_1002;  
27         return head_1002;  
28     }  
29  
30     for (int i = 1; temp_1002 != null && i < position_1002; i++) {  
31         prev_1002 = temp_1002;  
32         temp_1002 = temp_1002.next_1002;  
33     }  
34     if (temp_1002 != null)  
35         prev_1002.next_1002 = temp_1002.next_1002;  
36     else  
37         System.out.println("Data tidak ada");  
38     return head_1002;  
39 }  
40
```

Kode Program 2.12 – Blok Program Menghapus Node di Posisi Tertentu

Method ini digunakan untuk menghapus node pada posisi tertentu dalam linked list. Program menggunakan variabel **temp_1002** untuk menelusuri node dan **prev_1002** untuk menyimpan node sebelumnya.

Jika posisi yang dihapus adalah posisi pertama, maka head langsung dipindahkan ke node berikutnya. Namun jika posisi berada di tengah atau akhir linked list, program akan menelusuri node hingga mencapai posisi yang ditentukan. Setelah node ditemukan, pointer **next_1002** dari node sebelumnya akan diarahkan ke node setelah node yang dihapus sehingga node tersebut terlepas dari linked list.

Jika posisi yang dimasukkan melebihi jumlah node, program akan menampilkan pesan “Data tidak ada”.

2.4.4 Menampilkan Linked List

```
41 public static void printList_2511531002(NodeSLL_2511531002 head_1002) {  
42     NodeSLL_2511531002 curr_1002 = head_1002;  
43     while (curr_1002.next_1002 != null) {  
44         System.out.print(curr_1002.data_1002+"→");  
45         curr_1002 = curr_1002.next_1002;  
46     }  
47     if (curr_1002.next_1002 == null)  
48         System.out.print(curr_1002.data_1002);  
49     System.out.println();  
50 }  
51
```

Kode Program 2.13 – Blok Program Menampilkan Linked List

Method ini digunakan untuk menampilkan seluruh isi linked list secara berurutan mulai dari head hingga node terakhir. Program melakukan

penelusuran node menggunakan variabel **curr_1002** dan menampilkan data setiap node menggunakan format panah (-->).

2.4.5 Method Main

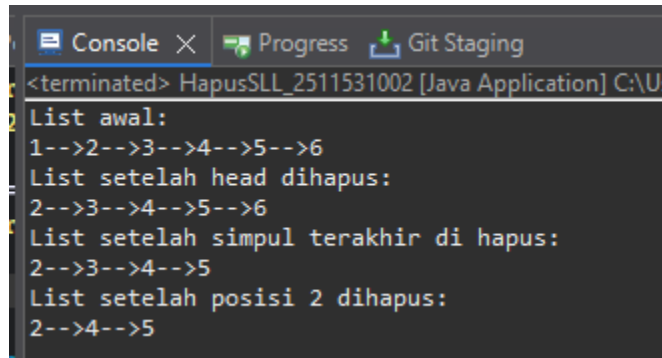
```
52 public static void main(String[] args) {
53     NodeSLL_2511531002 head_1002 = new NodeSLL_2511531002(1);
54     head_1002.next_1002 = new NodeSLL_2511531002(2);
55     head_1002.next_1002.next_1002 = new NodeSLL_2511531002(3);
56     head_1002.next_1002.next_1002.next_1002 = new NodeSLL_2511531002(4);
57     head_1002.next_1002.next_1002.next_1002.next_1002 = new NodeSLL_2511531002(5);
58     head_1002.next_1002.next_1002.next_1002.next_1002.next_1002 = new NodeSLL_2511531002(6);
59
60     System.out.println("List awal: ");
61     printList_2511531002(head_1002);
62
63     head_1002 = deleteHead_2511531002(head_1002);
64     System.out.println("List setelah head dihapus: ");
65     printList_2511531002(head_1002);
66
67     head_1002 = removeLastNode_2511531002(head_1002);
68     System.out.println("List setelah simpul terakhir di hapus: ");
69     printList_2511531002(head_1002);
70
71     int position_1002 = 2;
72     head_1002 = deleteNode_2511531002(head_1002, position_1002);
73     System.out.println("List setelah posisi 2 dihapus: ");
74     printList_2511531002(head_1002);
75 }
76 }
77 }
```

Kode Program 2.14 – Blok Program Main

Method ini merupakan method utama yang digunakan untuk menjalankan program. Pada method ini dibuat linked list awal yang terdiri dari beberapa node.

Selanjutnya program melakukan pengujian operasi penghapusan data, yaitu menghapus node pertama menggunakan method **deleteHead_2511531002()**, menghapus node terakhir menggunakan method **removeLastNode_2511531002()**, dan menghapus node pada posisi tertentu menggunakan method **deleteNode_2511531002()**.

Setelah setiap proses penghapusan dilakukan, hasil linked list akan ditampilkan menggunakan method **printList_2511531002()**, yang dimana hasil Output dari program adalah sebagai berikut:



```
Console X Progress Git Staging
<terminated> HapusSLL_2511531002 [Java Application] C:\U
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 2. 3 – Hasil Program Penghapusan Node pada SLL

2.5 Latihan Pasien dan Rumah Sakit

```
package pekan5_2511531002;

public class Pasien_2511531002 {
    String namaPasien_1002;
    String penyakit_1002;
    int antrian_1002;
    Pasien_2511531002 next_1002;

    // CONSTRUCTOR
    public Pasien_2511531002(String namaPasien_1002, String
    penyakit_1002, int antrian_1002) {
        this.namaPasien_1002 = namaPasien_1002;
        this.penakit_1002 = penyakit_1002;
        this.antrian_1002 = antrian_1002;
        this.next_1002 = null;
    }

    // GETTER
    public String getNamaPasien_1002() {
        return namaPasien_1002; }
    public String getPenyakit_1002() {
        return penyakit_1002; }
    public int getAntrian_1002() {
        return antrian_1002; }
    public Pasien_2511531002 getNext_1002() {
        return next_1002; }

    // SETTER
    public void setNamaPasien_1002(String namaPasien_1002) {
        this.namaPasien_1002 = namaPasien_1002; }
    public void setPenyakit_1002(String penyakit_1002) {
        this.penakit_1002 = penyakit_1002; }
    public void setAntrian_1002(int antrian_1002) {
        this.antrian_1002 = antrian_1002; }
    public void setNext_1002(Pasien_2511531002 next_1002) {
        this.next_1002 = next_1002; }
}
```

Kode Program 2.15 – Tugas Pasien_2511531002

Class ini digunakan untuk merepresentasikan node data pasien pada struktur Single Linked List. Pada class ini terdapat beberapa atribut yaitu **namaPasien_1002** untuk menyimpan nama pasien, **penyakit_1002** untuk menyimpan jenis penyakit pasien, **antrian_1002** untuk menyimpan nomor antrian, serta **next_1002** yang digunakan sebagai pointer untuk menghubungkan node ke data pasien berikutnya.

Class ini memiliki konstruktor **Pasien_2511531002()** yang digunakan untuk menginisialisasi data pasien saat objek dibuat. Konstruktor menerima parameter nama pasien, penyakit, dan nomor antrian, kemudian menyimpannya ke dalam atribut class. Nilai **next_1002** diatur **null** sebagai penanda bahwa node belum terhubung dengan node lain.

Selain itu, class ini juga memiliki method getter dan setter. Method getter digunakan untuk mengambil nilai dari atribut yang terdapat pada class, sedangkan method setter digunakan untuk mengubah nilai atribut sesuai kebutuhan program. Dengan adanya method getter dan setter, proses pengolahan data pasien menjadi lebih teratur dan mudah dikendalikan.

```
package pekan5_2511531002;

import java.util.Scanner;

public class RumahSakit_2511531002 {
    Pasien_2511531002 head_1002;
    int counter_1002;
    public RumahSakit_2511531002() {
        head_1002 = null;
        counter_1002 = 0;
    }

    // INSERT
    public void daftarPasien_1002(String namaPasien_1002,
String penyakit_1002) {
        counter_1002++;
        Pasien_2511531002 baru_1002 = new
Pasien_2511531002(namaPasien_1002, penyakit_1002, counter_1002);
        if (head_1002 == null) {
            head_1002 = baru_1002; // jika list kosong,
node baru menjadi head
        } else {
            Pasien_2511531002 temp_1002 = head_1002;
            while (temp_1002.getNext_1002() != null) {
                temp_1002 = temp_1002.getNext_1002();
            }
        }
    }
}
```

```

        temp_1002.setNext_1002(baru_1002); //
        menghubungkan node terakhir dengan node baru
    }
    System.out.println("Pasien berhasil didaftarkan!
    Nomor Antrian: " + counter_1002);
}

// DELETE HEAD
public void panggilPasien_1002() {
    if (head_1002 == null) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("Memanggil: " +
        head_1002.getNamaPasien_1002());
        head_1002 = head_1002.getNext_1002(); //
        menggeser head ke node berikutnya
    }
}

// DISPLAY
public void tampilkanAntrian_1002() {
    if (head_1002 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
    Pasien_2511531002 temp_1002 = head_1002;
    while (temp_1002 != null) {
        System.out.println(
            temp_1002.getAntrian_1002() + ". " +
            temp_1002.getNamaPasien_1002() + " - " +
            temp_1002.getPenyakit_1002()
        );
        temp_1002 = temp_1002.getNext_1002(); // pindah
        ke node berikutnya
    }
}

// SEARCH
public void cariPasien_1002(String namaPasien_1002) {
    Pasien_2511531002 temp_1002 = head_1002;
    boolean ketemu_1002 = false;

    while (temp_1002 != null) {
        if
        (temp_1002.getNamaPasien_1002().equalsIgnoreCase(namaPasien_1002)
        ) {
            System.out.println("Pasien ditemukan: " +
            temp_1002.getNamaPasien_1002());
            ketemu_1002 = true;
            break;
        }
        temp_1002 = temp_1002.getNext_1002(); // lanjut
        ke node berikutnya
    }

    if (!ketemu_1002) {
        System.out.println("Pasien tidak ditemukan!");
    }
}

```

```

    }
}

// CEK STATUS
public void cekStatus_1002() {
    if (head_1002 == null) {
        System.out.println("Antrian kosong!");
        return;
    }

    int jumlah_1002 = 0;
    Pasien_2511531002 temp_1002 = head_1002;

    // menghitung jumlah node dalam linked list
    while (temp_1002 != null) {
        jumlah_1002++;
        temp_1002 = temp_1002.getNext_1002();
    }

    System.out.println("Jumlah pasien: " + jumlah_1002);
    System.out.println("Pasien terdepan: " +
head_1002.getNamaPasien_1002());
}

public static void main(String[] args) {
    Scanner input_1002 = new Scanner(System.in);
    RumahSakit_2511531002 rs_1002 = new
RumahSakit_2511531002();
    int pilihan_1002;

    do {
        System.out.println("\n=== Antrian Rumah Sakit
NIM: 2511531002 ===");
        System.out.println("1. Daftarkan Pasien
(Insert)");
        System.out.println("2. Panggil Pasien (Delete
Head)");
        System.out.println("3. Tampilkan Antrian
(Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_1002 = input_1002.nextInt();
        input_1002.nextLine();

        switch (pilihan_1002) {
            case 1:
                System.out.print("Masukkan Nama Pasien:
");
                String namaPasien_1002 =
input_1002.nextLine();
                System.out.print("Masukkan Keluhan: ");
                String penyakit_1002 =
input_1002.nextLine();

                rs_1002.daftarPasien_1002(namaPasien_1002, penyakit_1002);

```

```

        break;
    case 2:
        rs_1002.panggilPasien_1002();
        break;
    case 3:
        rs_1002.tampilkanAntrian_1002();
        break;
    case 4:
        System.out.print("Cari nama: ");
        String cari_1002 = input_1002.nextLine();
        rs_1002.cariPasien_1002(cari_1002);
        break;
    case 5:
        rs_1002.cekStatus_1002();
        break;
    case 6:
        System.out.println("Keluar...");
        break;
    default:
        System.out.println("Pilihan tidak
valid!");
    }
    } while (pilihan_1002 != 6);
    input_1002.close();
}
}

```

Kode Program 2.16 – Tugas RumahSakit_2511531002

Program **RumahSakit_2511531002** merupakan program utama yang digunakan untuk menjalankan simulasi antrian pasien rumah sakit menggunakan struktur data Single Linked List. Program ini memanfaatkan class **Pasien_2511531002** sebagai node untuk menyimpan data pasien dan menggunakan Scanner untuk menerima input dari pengguna. Variabel **head_1002** digunakan sebagai penunjuk node pertama pada linked list, sedangkan **counter_1002** digunakan untuk memberikan nomor antrian secara otomatis.

Program memiliki beberapa method utama seperti **daftarPasien_1002()** untuk menambahkan pasien ke dalam antrian, **panggilPasien_1002()** untuk memanggil sekaligus menghapus pasien terdepan, **tampilkanAntrian_1002()** untuk menampilkan seluruh data pasien, **cariPasien_1002()** untuk mencari pasien berdasarkan nama, serta **cekStatus_1002()** untuk menampilkan jumlah pasien dan pasien terdepan pada antrian.

Pada method **main()**, program berjalan menggunakan perulangan **do-while** yang akan terus menampilkan menu hingga pengguna memilih keluar. Menu yang

tersedia meliputi pendaftaran pasien, pemanggilan pasien, menampilkan antrian, pencarian pasien, dan pengecekan status antrian. Struktur **switch-case** digunakan untuk menjalankan method sesuai pilihan pengguna. Program akan berhenti ketika pengguna memilih opsi keluar, kemudian Scanner ditutup. Hasil output program akan terlihat seperti berikut:

```

Console X Progress Git Staging
RumahSakit_2511531002 [Java Application] C:\Users\user\p2\poo

=== Antrian Rumah Sakit NIM: 2511531002 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Sujatmiko Arafuru
Masukkan Keluhan: Asam Lambung
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511531002 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Rizal Ekonomi
Masukkan Keluhan: Demam
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511531002 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Fuad Kicau Mania
Masukkan Keluhan: Diare
Pasien berhasil didaftarkan! Nomor Antrian: 3

Pasien berhasil didaftarkan! Nomor Antrian:
=== Antrian Rumah Sakit NIM: 2511531002 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil: Sujatmiko Arafuru

=== Antrian Rumah Sakit NIM: 2511531002 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
2. Rizal Ekonomi - Demam
3. Fuad Kicau Mania - Diare

```

```

=== Antrian Rumah Sakit NIM: 2511531002 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Cari nama: rizal ekonomi
Pasien ditemukan: Rizal Ekonomi

=== Antrian Rumah Sakit NIM: 2511531002 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien: 2
Pasien terdepan: Rizal Ekonomi

=== Antrian Rumah Sakit NIM: 2511531002 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
keluar...

```

Gambar 2. 5 – Hasil Program Antrian Loker

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Single Linked List merupakan struktur data dinamis yang terdiri dari node-node yang saling terhubung melalui pointer. Struktur ini memudahkan proses pengolahan data seperti penambahan, pencarian, penelusuran, dan penghapusan data tanpa harus menentukan ukuran data di awal.

Melalui implementasi program yang telah dibuat, operasi dasar pada Single Linked List dapat dijalankan dengan baik, seperti penambahan node di depan, belakang, dan posisi tertentu, pencarian data, serta penghapusan node. Selain itu, penerapan Single Linked List pada simulasi antrian rumah sakit menunjukkan bahwa struktur data ini dapat digunakan dalam pengolahan data secara dinamis dan terstruktur.

Dengan praktikum ini, pemahaman mengenai konsep dan implementasi Single Linked List menggunakan bahasa pemrograman Java menjadi lebih baik, khususnya dalam pengelolaan data yang saling terhubung melalui node.

DAFTAR PUSTAKA

- [1] Modul Praktikum Struktur Data Pekan 5 – Single Linked List.